

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)»
(МАИ)

ОТЧЁТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
по теме:
РАЗРАБОТКА РОБОТИЗИРОВАННОЙ РУКИ С ДИСТАНЦИОННЫМ
УПРАВЛЕНИЕМ ПО WI-FI И СИСТЕМОЙ РЕГИСТРАЦИИ ДЕЙСТВИЙ

Руководитель НИР,
к.т.н, зав. кафедрой 307

подпись, дата

Васильев Ф.В.

Москва 2026

СОДЕРЖАНИЕ

Введение	3
1 Подготовительный этап	5
2 Сборка механической части	7
3 Электрическая схема подключения	12
3.1 Схема подключения сервоприводов	13
4 Программная часть проекта	14
4.1 Веб-интерфейс управления роборукой	14
4.2 Стоимость проекта	15
Приложение А Листинг кода микроконтроллера Arduino	17
Приложение Б Серверное приложение server.py	20

ВВЕДЕНИЕ

В рамках данного проекта была разработана и собрана роботизированная рука с дистанционным управлением через сеть Wi-Fi. Идея создания устройства возникла из интереса к робототехнике, программированию микроконтроллеров и взаимодействию аппаратной и программной частей в единой системе. Проект позволил на практике объединить знания из нескольких областей: электроники, программирования микроконтроллеров, сетевых технологий и веб-разработки.

За основу был взят готовый комплект роботизированной руки, включающий механические детали конструкции и необходимые элементы крепления. Использование готового набора позволило сосредоточить основное внимание на сборке устройства, его подключении, настройке и разработке программной части проекта.

Основной задачей являлось создание роботизированной руки, которой можно управлять удалённо через веб-интерфейс. В качестве исполнительного механизма используется сервопривод, обеспечивающий движение захвата. Управление осуществляется через веб-страницу с кнопками, доступную по локальной сети с компьютера или мобильного устройства.

Аппаратная часть проекта построена на базе платы Arduino Uno и Wi-Fi-модуля на ESP12S. При нажатии пользователем кнопок на веб-странице серверное приложение передаёт команду по сети на Arduino, после чего микроконтроллер изменяет положение сервопривода и приводит механизм руки в действие.

Дополнительно в проекте реализована система регистрации действий пользователя. Каждая отправленная команда автоматически сохраняется в базе данных с отметкой времени выполнения. Благодаря этому можно отслеживать историю работы устройства и контролировать последовательность выполненных действий.

Разработанная система демонстрирует практическое применение микроконтроллерных технологий, беспроводной передачи данных и веб-интерфейсов управления. Проект показывает, каким образом механическое

устройство может быть интегрировано в современную программно-аппаратную систему с возможностью удалённого управления и ведения журнала событий.

Посмотреть на работу роборуки можно по ссылке в ВК Видео

1 Подготовительный этап

Перед началом сборки роботизированной руки был проведён анализ возможных вариантов реализации проекта и подбор необходимых компонентов. В качестве основы было принято решение использовать готовый комплект роботизированной руки, поскольку такой подход позволял сократить время на изготовление механической части и сосредоточиться на изучении принципов работы системы управления, программирования и сетевого взаимодействия.

Таблица 1 — Список компонентов и их назначение

Наименование	Значение
Роботизированная рука (комплект деталей)	Механическая часть устройства
Arduino Uno	Управление исполнительными механизмами
Osoyoo Uart ESP12S V1.1	Беспроводная передача данных
Сервоприводы	Приведение механизма руки в движение
Соединительные провода	Подключение электронных компонентов
USB-кабель	Программирование и питание Arduino

На этапе подготовки также было выбрано программное обеспечение, необходимое для разработки проекта. Для программирования микроконтроллера использовалась среда Arduino IDE. Для создания веб-интерфейса управления и организации обмена командами с устройством применялся язык программирования Python и веб-фреймворк Flask. Для хранения истории действий была выбрана встроенная база данных SQLite, не требующая установки дополнительного серверного программного обеспечения.

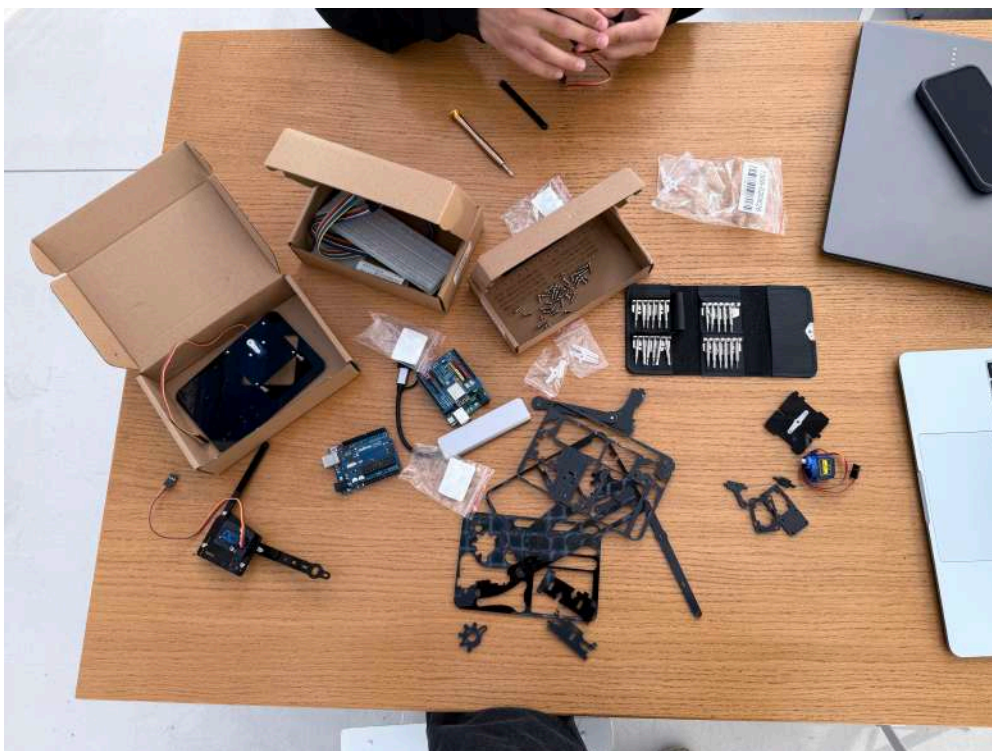


Рисунок 1 — Комплектующие проекта на подготовительном этапе перед началом сборки роботизированной руки

После подготовки всех компонентов была выполнена проверка их работоспособности, а также изучена документация по подключению Wi-Fi-модуля ESP12S к плате Arduino Uno и взаимодействию устройств по локальной сети.



Рисунок 2 — Проверка работоспособности Arduino Uno через Serial Monitor в Arduino IDE

По итогам подготовительного этапа были собраны все необходимые аппаратные и программные средства для дальнейшей сборки и настройки роботизированной руки.

2 Сборка механической части

После завершения подготовительного этапа была выполнена сборка механической части роботизированной руки. В качестве основы использовался готовый комплект деталей, крепёжные изделия и сервоприводы, представленные в таблице 2.

Таблица 2 — Состав комплекта роботизированной руки

Наименование	Количество
Основание	1
Опорная пластина	1
Основа левой руки	1
Основа правой руки	1
Рычаг левой руки	1
Рычаг правой руки	1
Траверса основания манипулятора	1
Соединительное ребро жёсткости	2
Балка левого запястья	1
Балка правого запястья	1
Параллельная балка	1
Параллельное крепление	2
Левый захват	1
Правый захват	1
Левое крепление запястья	1
Правое крепление запястья	1
Нижнее крепление сервопривода	1
Верхнее крепление сервопривода	1
Приводной рычаг	2
Коннектор	1
Прокладка	3
Сервопривод	4
Винт М3×6 мм	9
Винт М3×8 мм	15
Винт М3×10 мм	6
Винт М3×12 мм	1
Винт М3×20 мм	4
Гайка М3	10

Сборка началась с подготовки деталей и сортировки крепёжных элементов. Для удобства монтажа были изучены схема сборки и назначение отдельных компонентов конструкции. Особое внимание уделялось правильному расположению подвижных элементов, поскольку от качества сборки напрямую зависит работоспособность механизма. Обзорно ознакомиться с составом комплекта можно в таблице 2 или на рисунке 1.

На следующем этапе были собраны основные узлы роботизированной руки: основание, рычажные механизмы и захват. Детали соединялись между собой с помощью винтов и пластиковых креплений, входящих в комплект поставки. После установки механических элементов были смонтированы сервоприводы, обеспечивающие движение подвижных частей конструкции. Компоновка деталей осуществлялась поэтапно. На рисунке 3 представлена первая часть сборки механизма хвата с сервоприводом SG-90.



Рисунок 3 — Первая часть сборки хвата роборуки

Особую сложность представляла вторая часть ввиду нестандартной (для нас) связи сервы с шестернями – рисунок 4.

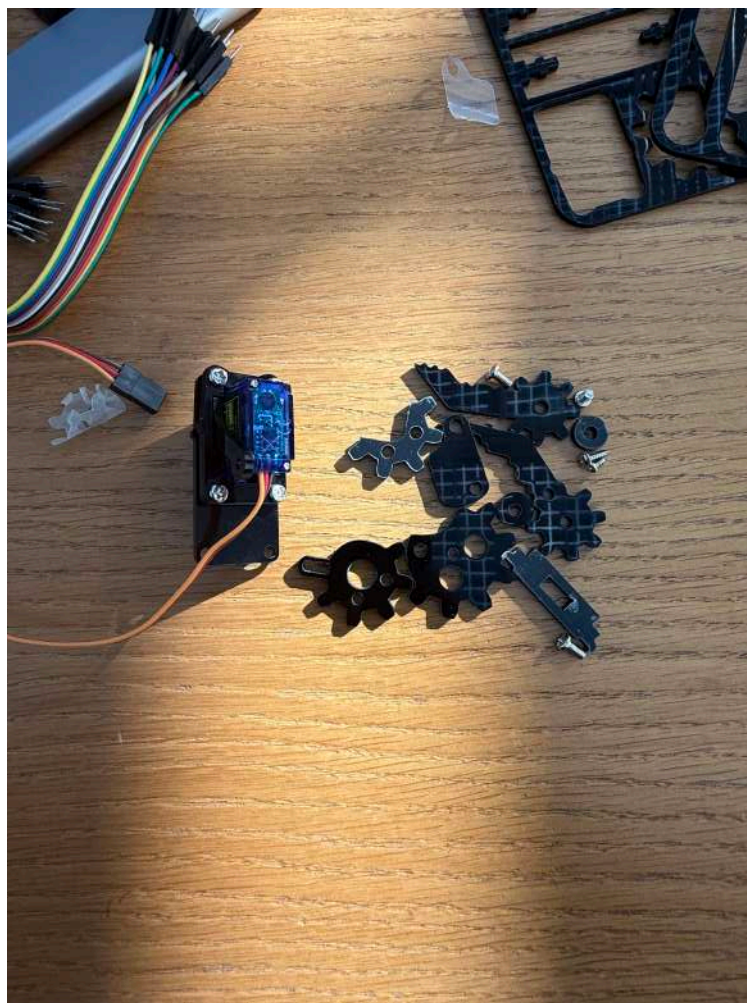


Рисунок 4 — Вторая часть сборки хвата роборуки

В процессе сборки выполнялась периодическая проверка свободы перемещения элементов и люфта конструкции. Это позволило своевременно обнаружить возможные перекосы конструкции и исключить чрезмерную нагрузку на сервоприводы или усталость деталей при дальнейшей эксплуатации устройства.



Рисунок 5 — Клешня, осуществляющая сжатие-разжатие

По завершении монтажа была получена полностью собранная механическая конструкция роботизированной руки, готовая к подключению электронных компонентов и последующей настройке системы управления.

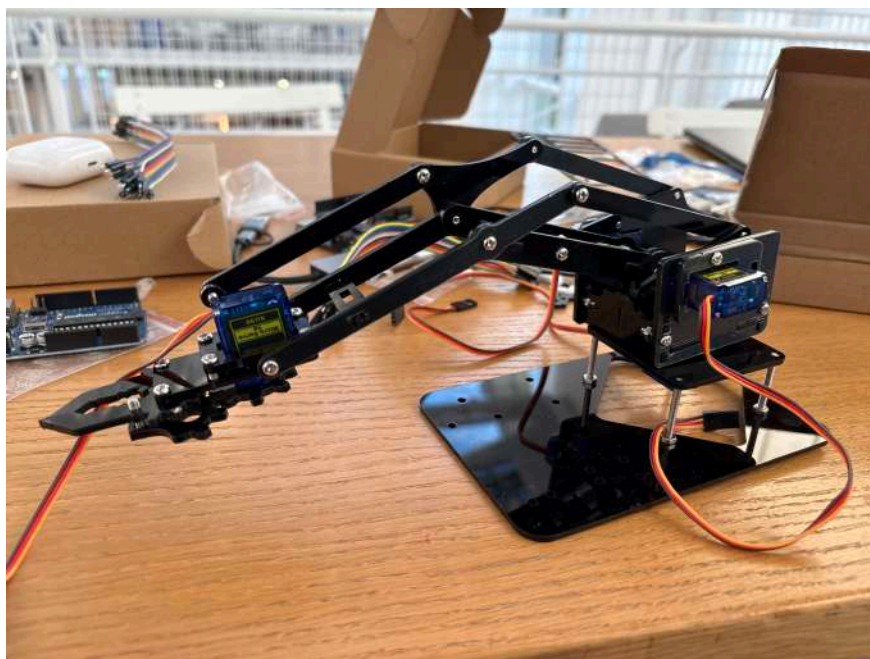


Рисунок 6 — Итоговый вид модели роборуки с четырьмя работающими сервоприводами

Проверка показала, что все подвижные элементы перемещаются свободно, а механизм захвата корректно реагирует на изменение положения сервопривода. Результатом данного этапа стала собранная механическая платформа роботизированной руки, которая в дальнейшем использовалась для подключения платы Arduino Uno, Wi-Fi-модуля ESP12S и разработки системы дистанционного управления.

3 Электрическая схема подключения

После сборки механической части роботизированной руки была выполнена установка и подключение электронных компонентов системы управления. В качестве основного управляющего устройства использовалась плата Arduino Uno. Для обеспечения удалённого управления через локальную сеть к микроконтроллеру был подключён Wi-Fi-модуль Osoyoo Uart ESP12S V1.1.

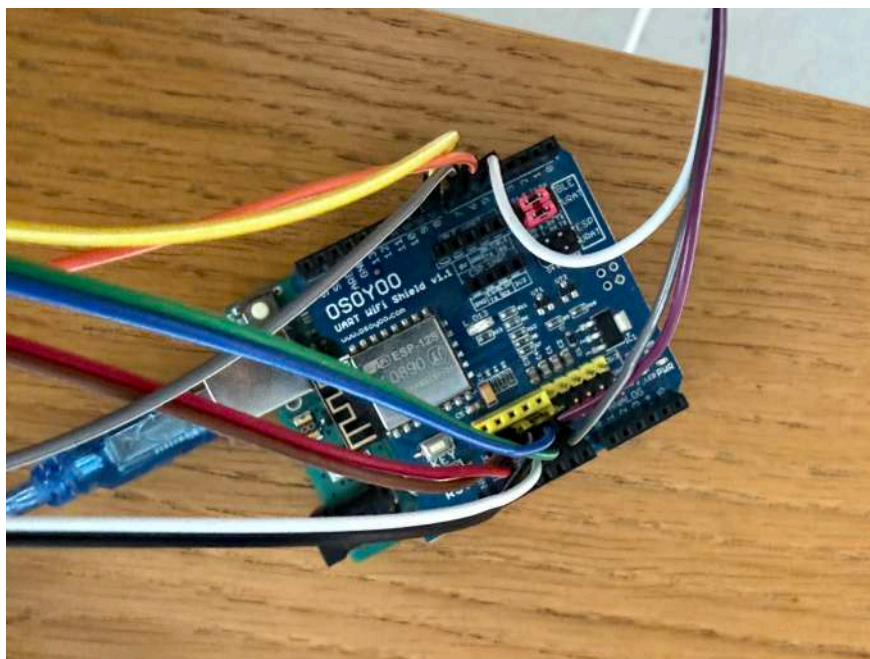


Рисунок 7 — Установленный WI-FI-модуль Osoyoo Uart ESP12S V1.1 на плату Arduino Uno

Для организации удалённого управления роботизированной рукой использовался Wi-Fi-модуль Osoyoo Uart ESP12S V1.1, установленный на плату Arduino Uno. Модуль обеспечивает подключение устройства к локальной сети и позволяет принимать управляющие команды, передаваемые с веб-интерфейса. Полученные команды обрабатываются микроконтроллером и используются для управления исполнительными механизмами роботизированной руки.

После подключения Wi-Fi-модуля были выполнены соединения между платой Arduino Uno и сервоприводами. Для распределения питания использовалась макетная плата, позволяющая подключить все сервоприводы к общим линиям питания и земли.

3.1 Схема подключения сервоприводов

Управляющие сигналы на сервоприводы подаются с цифровых выводов D5–D8 платы Arduino Uno. Питание исполнительных механизмов осуществляется от линии +5В, а общий провод подключён к выводу GND. Такая схема обеспечивает одновременную работу всех сервоприводов и упрощает подключение компонентов системы.

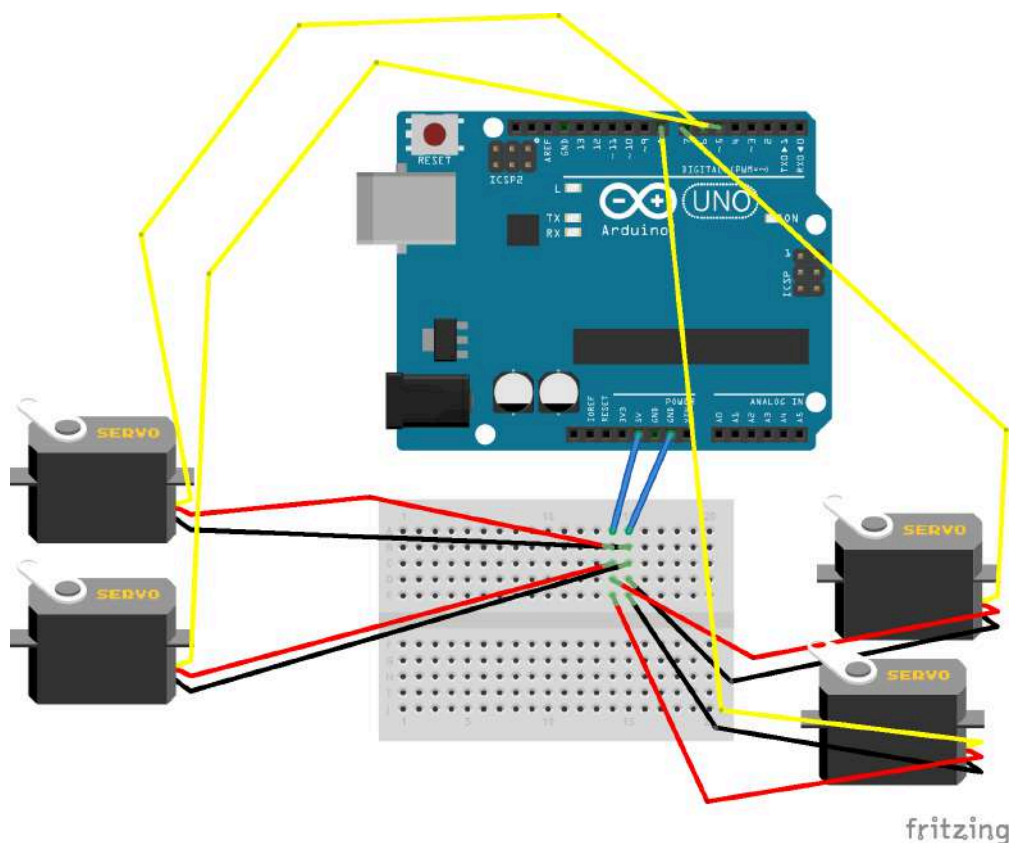


Рисунок 8 — Схема подключения сервоприводов к Arduino Uno

Таблица 3 — Подключение сервоприводов к цифровым выводам Arduino Uno

Элемент	Подключение
Сервопривод №1	D5
Сервопривод №2	D6
Сервопривод №3	D7
Сервопривод №4	D8
Питание сервоприводов	5V
Общий провод	GND

4 Программная часть проекта

Для управления роботизированной рукой было разработано программное обеспечение, состоящее из программы для Arduino Uno и серверного приложения на языке Python.

Arduino осуществляет управление сервоприводами и выполняет поступающие команды. Передача команд осуществляется через Wi-Fi-модуль Osoyoo Uart ESP12S V1.1. Исходный код представлен в приложении А.

Для взаимодействия пользователя с устройством разработан веб-интерфейс, позволяющий отправлять команды через браузер. Серверная часть приложения написана на языке Python с использованием фреймворка Flask. Исходный код представлен в приложении Б.

Дополнительно в проекте реализована система регистрации действий пользователя. Все отправленные команды автоматически сохраняются в базе данных SQLite с указанием времени выполнения операции.

4.1 Веб-интерфейс управления роборукой

Реализована страница по ip-адресу сервера локальной сети на порте 5050.

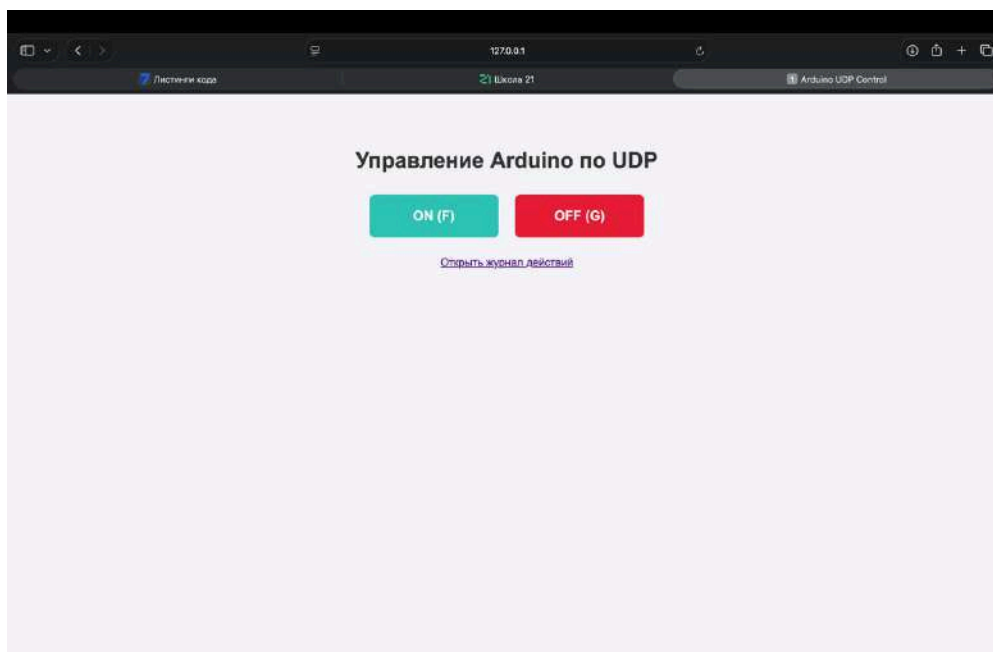
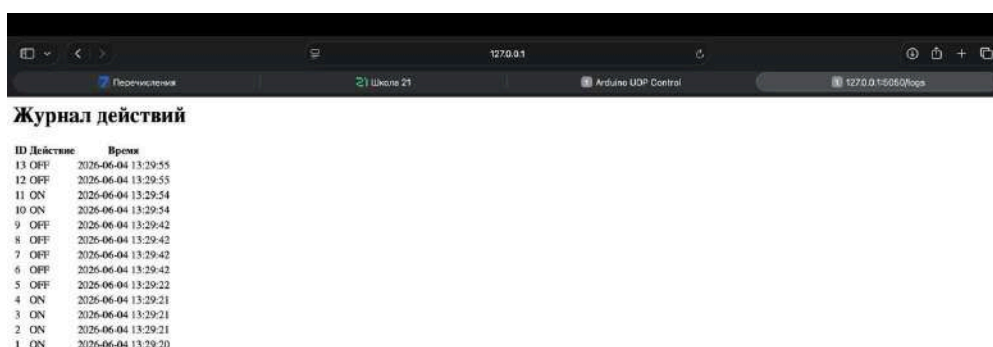


Рисунок 9 — Скриншот с сайта

Интерфейс представляет собой страницу, на которой расположены две кнопки: ON (F), OFF(G). По нажатию на первую кнопку клешня роборуки раскрывается, по нажатию на вторую закрывается. История о взаимодействии с кнопками хранится в файле logs.db в таблице logs. Структура таблицы включает следующие поля:

- id — уникальный идентификатор записи;
- action — выполненное действие (ON или OFF);
- timestamp — дата и время выполнения команды.



ID Действие	Время
13 OFF	2025-06-04 13:29:55
12 OFF	2025-06-04 13:29:55
11 ON	2025-06-04 13:29:54
10 ON	2025-06-04 13:29:54
9 OFF	2025-06-04 13:29:42
8 OFF	2025-06-04 13:29:42
7 OFF	2025-06-04 13:29:42
6 OFF	2025-06-04 13:29:42
5 OFF	2025-06-04 13:29:22
4 ON	2025-06-04 13:29:21
3 ON	2025-06-04 13:29:21
2 ON	2025-06-04 13:29:21
1 ON	2025-06-04 13:29:20

Рисунок 10 — Пример вкладки с историей взаимодействий

Принцип работы системы логирования заключается в автоматическом сохранении информации после каждой команды, отправленной через веб-интерфейс. При нажатии пользователем кнопки управления серверное приложение передаёт команду на Arduino Uno и одновременно создаёт новую запись в базе данных.

4.2 Стоимость проекта

Таблица 4 — Расчет стоимости аппаратной и программной части

Наименование компонента / ПО	Примерная стоимость (руб.)
Готовый комплект деталей роботизированной руки (механика)	1 500
Микроконтроллер Arduino Uno	600

Наименование компонента / ПО	Примерная стоимость (руб.)
Wi-Fi-модуль Osoyoo Uart ESP12S V1.1 (на базе ESP8266)	550
Сервоприводы SG-90 / MG-90S (4 шт. × 250 руб.)	1 000
Кабель USB (для питания и прошивки)	150
Итоговая стоимость проекта:	3 800

ПРИЛОЖЕНИЕ А

Листинг кода микроконтроллера Arduino

```
#include <Servo.h>
#include "WiFiEsp.h"
#include <WiFiEspUdp.h>
#include "SoftwareSerial.h"

#define CLAW_PIN 7

// Настройка программного Serial для связи с ESP8266 (D4 -> TX, D5 ->
RX)
SoftwareSerial softserial(4, 5);

char ssid[] = "MERCUSYS_D2B9";
char pass[] = "221351518";

int status = WL_IDLE_STATUS;
unsigned int local_port = 8888;

char packetBuffer[255];
Servo claw;
WiFiEspUDP Udp;

void setup()
{
    Serial.begin(9600);

    softserial.begin(115200);
    softserial.write("AT+CI0BAUD=9600\r\n");
    softserial.write("AT+RST\r\n");
    delay(1000);
    softserial.begin(9600);

    Serial.println("\n--- Старт устройства ---");
    WiFi.init(&softserial);

    if (WiFi.status() == WL_NO_SHIELD) {
        Serial.println("Ошибка: Модуль не найден!");
        while (true);
    }
}
```

```

while (status != WL_CONNECTED) {
    Serial.print("Подключение к SSID: ");
    Serial.println(ssid);
    status = WiFi.begin(ssid, pass);
}

Serial.println("Успешно подключено к Wi-Fi!");
Serial.print("IP-адрес твоей платы: ");
Serial.println(WiFi.localIP());

Udp.begin(local_port);

claw.attach(CLAW_PIN);
claw.write(90);
Serial.println("Клешня готова. Ожидание UDP команд...\n");
}

void loop()
{
    int packetSize = Udp.parsePacket();

    if (packetSize) {
        int len = Udp.read(packetBuffer, 255);
        Udp.flush();

        if (len > 0) {
            packetBuffer[len] = 0;
        }

        Serial.print("[UDP] Получено: ");
        Serial.println(packetBuffer);

        if (strcmp(packetBuffer, "F") == 0) {
            Serial.println("Action: ON -> claw.write(170)");
            claw.write(170);
        }

        else if (strcmp(packetBuffer, "G") == 0) {
            Serial.println("Action: OFF -> claw.write(80)");
            claw.write(80);
        }
    }
}

```

```
    }  
    memset(packetBuffer, 0, sizeof(packetBuffer));  
    Serial.println("-----");  
  }  
}
```

Листинг A.1 — Листинг кода микроконтроллера Arduino

ПРИЛОЖЕНИЕ Б

Серверное приложение server.py

```
import socket
import time
import sqlite3
from datetime import datetime
from flask import Flask, render_template_string

app = Flask(__name__)

ARDUINO_IP = "192.168.0.105"
ARDUINO_PORT = 8888

DB_NAME = "logs.db"

def init_db():
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            action TEXT NOT NULL,
            timestamp TEXT NOT NULL
        )
    """)

    conn.commit()
    conn.close()

def save_log(action):
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()

    cur.execute(
        "INSERT INTO logs (action, timestamp) VALUES (?, ?)",
        (action, datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    )
```

```
conn.close()
```

```
print(f"[ОШИБКА] Не удалось отправить UDP: {e}")
```

```
font-weight: bold;
```

```

        color: white;
        border: none;
        border-radius: 8px;
        cursor: pointer;
        text-decoration: none;
        transition: 0.2s;
    }

    .btn-on {
        background-color: #2ec4b6;
    }

    .btn-on:hover {
        background-color: #25a195;
    }

    .btn-off {
        background-color: #e71d36;
    }

    .btn-off:hover {
        background-color: #c0182d;
    }

    iframe {
        display: none;
    }

    table {
        margin: 30px auto;
        border-collapse: collapse;
    }

    th, td {
        border: 1px solid #ccc;
        padding: 8px 15px;
    }
</style>
</head>
<body>

<h1>Управление Arduino по UDP</h1>

```

```

<a href="/action/on" target="hidden-form" class="btn btn-on">ON (F)</a>
<a href="/action/off" target="hidden-form" class="btn btn-off">OFF
(G)</a>

<br><br>

<a href="/logs" target="_blank">Открыть журнал действий</a>

<iframe name="hidden-form"></iframe>

</body>
</html>
"""

```

```

@app.route("/")
def index():
    return render_template_string(HTML_TEMPLATE)

```

```

@app.route("/action/on")
def action_on():
    send_udp("F")
    save_log("ON")
    return "OK"

```

```

@app.route("/action/off")
def action_off():
    send_udp("G")
    save_log("OFF")
    return "OK"

```

```

@app.route("/logs")
def logs():
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()

    cur.execute(
        "SELECT id, action, timestamp FROM logs ORDER BY id DESC"
    )

```



```

rows = cur.fetchall()
conn.close()

html = """
<h1>Журнал действий</h1>

<table>
    <tr>
        <th>ID</th>
        <th>Действие</th>
        <th>Время</th>
    </tr>
"""

for row in rows:
    html += f"""
    <tr>
        <td>{row[0]}</td>
        <td>{row[1]}</td>
        <td>{row[2]}</td>
    </tr>
    """

html += "</table>"

return html

if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5050, debug=True)

```

Листинг Б.1 — Серверное приложение server.py